

Data Structures and Algorithms

Lecture 1: C Programming Language

Data Science

Beijing Normal-Hong Kong Baptist University

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- Structure
- Dynamic Memory Allocation
- Recursion

Introduction to C

- C is a **structured programming language**
- A program is made up of functions
 - One **main** function (mandatory)
 - Sub-functions (optional)
- Created by Dennis M. Ritchie in 1972
- ANSI (American National Standards Institute)
 - First edition: ANSI C 1989
 - Second edition: ANSI C 1999
 - Other editions: C11, C17

First Program in C

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

What does this program do?

Preprocessor

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

C preprocessor directive

- In general, C compilers perform some preparatory work on source code before compiling; this is termed **preprocessing**
- The effect of **#include <stdio.h>** is the same as if you had typed the entire contents of the **stdio.h** file into your file at the point where the **#include** line appears

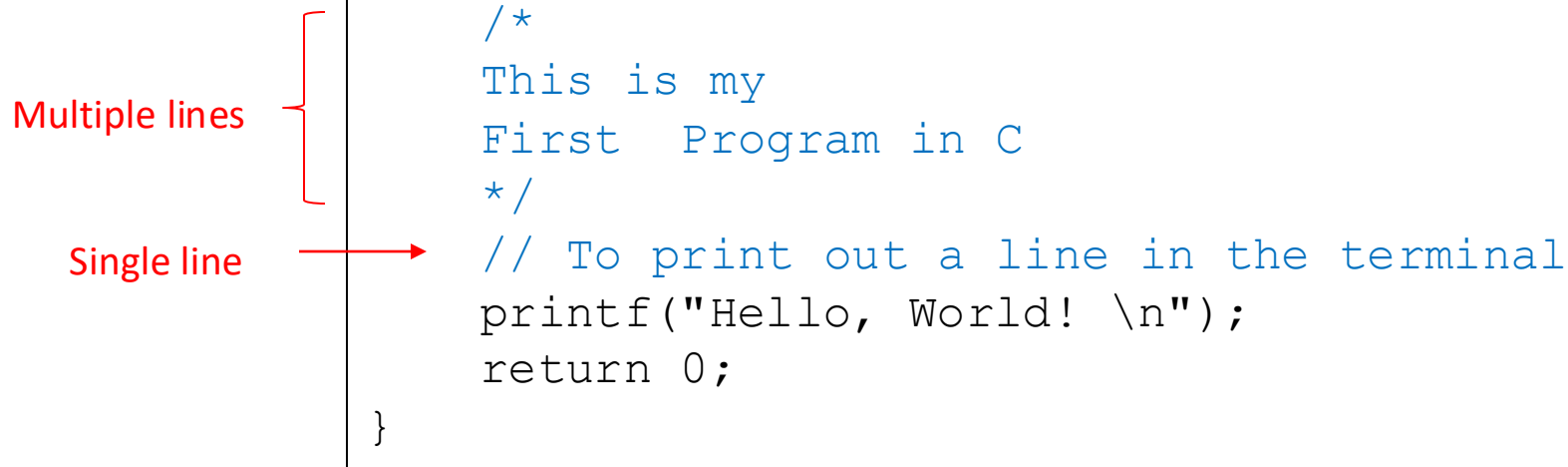
Header

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

Header

- The **stdio.h** file is supplied as part of all C compiler packages. It contains information about input and output functions, such as **printf()**, for the compiler to use.
- C people call a collection of information that goes at the top of a file a **header**. C implementations typically come with several header files.

Comments



The diagram illustrates C code comments. A red bracket on the left groups three lines of code under the label "Multiple lines":
/*
This is my
First Program in C
*/
A red arrow points from the label "Single line" to the following line of code:
// To print out a line in the terminal
The rest of the code is:
printf("Hello, World! \n");
return 0;
}

```
#include <stdio.h>
int main()
{
    /*
    This is my
    First Program in C
    */
    // To print out a line in the terminal
    printf("Hello, World! \n");
    return 0;
}
```

Comments

- A comment is used to explain some parts in the program
- Two ways to insert comments in C
 1. **//**(double slash), only comment one line
 2. **/* */** , used to comment multiple lines

Main Function

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

Main function

- All the programs written in C must have a main function
 - **main**: function name
 - **int**: function return type
- The part between the first '{' and the last '}' is called **body of function**

Statement

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

Statement

- A statement is an instruction telling a computer what to do
- A **simple statement** ends with ‘;’
- A **compound statement** will include a sequence statements or conditions

The printf() Function



The diagram shows the C code `printf( "\n Hello World! \n" );` enclosed in a rectangular box. Three arrows point to specific parts of the code: one points to the opening parenthesis '(', another points to the first backslash-newline sequence `\n`, and a third points to the second backslash-newline sequence `\n`. Each arrow is accompanied by the text 'Start at a new line'.

```
printf( "\n Hello World! \n" );
```

- This statement uses a standard C function called **printf**
- **printf** is the function name
- Output information to screen in the described format

Output:

Hello World!

How Does A Program Work (for a single source file)

If the output is incorrect (Possible **Bugs**)

If the code has syntax errors

Edit

.c file

Compile& Build

.exe file

Run and Test

```
#include <stdio.h>
int main()
{
    /* First Program in C */
    printf("Hello, World! \n");
    return 0;
}
```

Source file
(sample.c)

00001111	11111111
01011111	00010101
01010101	01010010
01011111	00010101

Executable file
(sample.exe)

Exercise

What is the output of this program?

```
/* just an example */  
#include<stdio.h>  
int main() {  
    printf("Hello World\n");  
    printf("Hello World\n");  
    printf("Hello World\n");  
    return 0;  
}
```

```
Hello World  
Hello World  
Hello World
```

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- Structure
- Dynamic Memory Allocation
- Recursion

Primary Data Types

- `int`
 - Used to express the integer type. The biggest integer that can be expressed in a computer depends on the host computer (32 bits or 64 bits)
- `char`
 - Used to express the single characters. Each character corresponds to an integer between 0 and 127
- `float`
 - Real number (single precision float point)
- `double`
 - Real number (double precision float point)

The int Type

- The int type is a signed integer.
 - It must be an integer and it can be positive, negative, or zero.
- Can be expressed in
 - Decimal (base-10)
 - Hexadecimal (base-16)
 - Octal (base-8)

Other int Types

- C offers three adjective keywords to modify the basic integer type: *short* , *long* , and *unsigned* .
 - The type *short int* or *short* may use less storage than *int* , thus saving space when only small numbers are needed (signed type).
 - The type *long int* , or *long* , may use more storage than *int* , thus enabling you to express larger integer values. (signed type)
 - The type *long long int* , or *long long* may use more storage than *long* (at least 64 bits, signed type)
 - The type *unsigned int* , or *unsigned* , is used for variables that have only nonnegative values
 - The keyword *signed* can be used with any of the signed types to make your intent explicit. For example, *short* , *short int* , *signed short* , and *signed short int* are all names for the same type.

Other int Types (Cont.)

- Other integer types are declared in the same manner as *int* type.

```
long int estine;  
long johns;  
short int erns;  
short ribs;  
unsigned int s_count;  
unsigned players;  
unsigned long headcount;  
long long ago;
```

- The most common practice today on personal computers is to
 - Set up *long long* as 64 bits
 - Set up *long* as 32 bits, *short* as 16 bits
 - Set up *int* as either 16 bits or 32 bits, depending on machine's natural word size

The char Type

- The *char* type is used for storing characters such as letters and punctuation marks
 - But technically, it is an integer type.
- To handle characters, the computer uses a numerical code in which certain integers represent certain characters.

The char Type

- The *char* type is used for storing characters such as letters and punctuation marks
 - But technically, it is an integer type.
- To handle characters, the computer uses a numerical code in which certain integers represent certain characters.
- The char type is typically defined as an 8-bit unit of memory.
 - C enabled you to use the keywords *signed* and *unsigned* with *char*

The float and double Type

- In C, floating-point numbers are called type *float* , *double* , or *long double*.
- Float
 - 4 bytes, e.g., 1.2, 2.5e8
 - IEEE 754 standard: 1 bit sign, 8 bits exponent, 23 bits mantissa
 - $1.2 \times 10^{-38} \sim 3.4 \times 10^{38}$ (1.2e-38 ~ 3.4e38)
- Double
 - 8 bytes
 - IEEE 754 standard: 1 bit sign, 10 bits exponent, 53 bits mantissa
 - $2.2 \times 10^{-308} \sim 1.8 \times 10^{308}$ (2.2e-308 ~ 1.8e308)

Identifiers (Variable Names)

- An **identifier** consists of a letter or underscore followed by any sequence of letters, digits or underscores
 - E.g., `_ls`, `This_ls`, `A12`, `a23`
- Names are **case-sensitive!** The following are unique identifiers:
 - `Hello`, `hello`,
 - `whoami`, `whoAMI`, `WhoAml`
- C **keywords** (**reserved words**) cannot be used as identifiers.
- Choose identifiers that are meaningful and easy to remember.

Exercises

- Are these the valid variable names?
 - `_123`
 - `_abc`
 - `Example`
 - `Abc123`
 - `unsigned`
 - `int`
 - `a%b`
 - `2example`
 - `Xx`

Declaration and Assignment

- Every variable used in a program must declare its type
 - Format: **TYPE** **variable_name_list**;
 - E.g.,
 - `int i;`
 - `float f;`
 - `double area;`
 - `unsigned int number;`
 - `int number, index, grade;`

Declaration and Assignment (Cont.)

- The variables can be assigned values using the assignment operator
 - Format: `variable_name = value ;`
 - E.g.,
 - `int i = 10;`
 - `double f = 1.2;`
 - `float area = 6.28f ;`
 - `char c = 'a' ;`

Type Conversion

- C allows for conversions between the basic types, implicitly or explicitly.
- Explicit conversion uses the *cast* operator.
- If the compiler expects one type at a position, but another type is provided, then implicit conversion occurs.

```
int x = 10;
float y = 3.14, z = 3.14;
y = (float) x;      // y = 10.0
x = (int) z;        // x = 3
x = (int) (-z);     // x = -3, rounded approaching zero
y = x;              // y = -3.0, implicit conversion
```

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- Structure
- Dynamic Memory Allocation
- Recursion

The printf Function

- Output information to screen in the described format
- Format:
 - `printf(format_string, arg1, arg2, ..., argn)`

```
printf("The results are %d and %d\n", a, b);
```



Format_string - included in a pair of " "

arg1

arg2

The printf Function (Cont.)

- Format string contains (optional):
 - **literal text**: printed without any variation
 - **escape sequences**: preceded by `\`, used to print special characters
 - **conversion specifiers** : starts with `%` and ends with output conversion characters, indicating that the value of a variable is to be displayed at this location.
- Format string should contain a conversion specifier for each item to be printed
 - The variables to be printed must appear as `arg1`, `arg2`,in the order that they appear in the format string

The printf Function (Cont.)

The diagram shows the code `printf("The results are %d and %d\n", a, b);` enclosed in a box. Below the box, several annotations with arrows point to specific parts of the code:
 - A red arrow points from the text "literal text" to the string "The results are".
 - A red arrow points from the text "conversion specifier" to the first `%d`.
 - A red arrow points from the text "literal text" to the string " and ".
 - A red arrow points from the text "conversion specifier" to the second `%d`.
 - A blue arrow points from the text "escaped sequences" to the `\n` sequence.
 - Two curved red arrows originate from the first and second `%d` conversion specifiers and point to the variables `a` and `b` respectively, indicating argument passing.

```
printf("The results are %d and %d\n", a, b);
```

literal text

conversion specifier

literal text

conversion specifier

escaped sequences

If `a = 10` and `b = 20`, we have the following output:

```
The results are 10 and 20
```

Conversion Specifiers

Conversion Specifier

Meaning

<code>%c</code>	Single character
<code>%d</code>	Signed decimal integer
<code>%x</code>	Hexadecimal number
<code>%f</code>	Decimal floating point number
<code>%lf</code>	Decimal double number
<code>%e</code>	Floating point in “scientific notation”
<code>%s</code>	Character string (more on this later)
<code>%u</code>	Unsigned decimal integer
<code>%%</code>	Just print a % sign
<code>%ld, %lld</code>	long, and long long

Exercises

- What is the output of the following program?

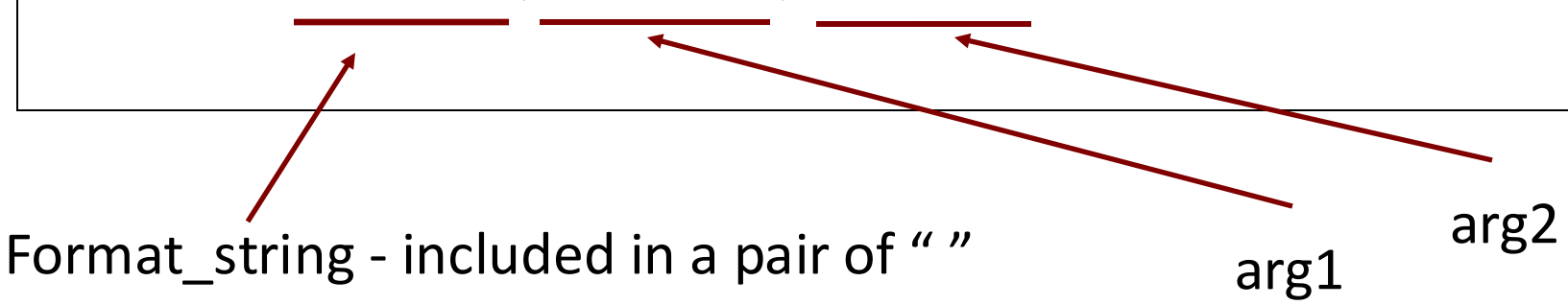
```
#include <stdio.h>
int main()
{
    char c, d;
    float f;
    c = 'd';
    d = 97;
    f = 23.5f;
    printf("c = %c, d = %c\nf = %f, f = %e", c, d, f, f);
}
```

```
c = d, d = a
f = 23.500000, f = 2.350000e+01
```

The scanf Function

- Read the input from keyboard
- Format:
 - `scanf(format_string, arg1, arg2, ..., argn);`

```
scanf("%d %f", &value, &ratio);
```



Format_string - included in a pair of “ ”

arg1

arg2

&: means “address of”. Will be explained later.

The scanf Function

- Read the input from keyboard
- Format:
 - `scanf(format_string, arg1, arg2, ..., argn);`

```
scanf("%d %f", &value, &ratio);
```

Input:

10

15.9

Exercise

- What is the output of the following program

```
#include <stdio.h>
int main() {
    int percentage, x;
    char ch1, ch2;
    scanf("%d%c%c%d", &percentage, &ch1, &ch2, &x);
    printf("%d%% %c%c %d is %f\n", percentage, ch1,
        ch2, x, 1.0 * x *percentage / 100);
    return 0;
}
```

- What does this program do?
- What is the output if the input is **25of60**

Exercise

- What is the output of the following program

```
25of60  
25% of 60 is 15.000000
```

Note: In new versions of C compilers such as VS 2019, using “scanf” may results errors or warnings:

C4996 'scanf': This function or variable may be unsafe. Consider using scanf_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See online help for details.

This is because there are potential safety issues with this function “scanf”. But for now it doesn’t matter for us.

So the easiest solution is to put a MACRO line in the **very beginning** of your code:

```
#define _CRT_SECURE_NO_WARNINGS ←  
#include <stdio.h>  
  
int main() {  
...  
}
```

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- **Function**
- Array and Pointers
- String
- Structure
- Dynamic Memory Allocation
- Recursion

Structured Programming

- **Structured Programming** is a disciplined software development approach
 - C programming language is a structured programming language
 - C's design philosophy is to use functions as building blocks
 - One main function (mandatory)
 - Sub-functions (optional)

Function

- A complex problem is often easier to solve by dividing it into several smaller sub-problems (tasks)
 - These smaller sub-problems are sometimes made into packaged code chunks called functions in C
- To write a function, we normally need to specify
 - function **declaration** (prototype)
 - function **definition**
- A function definition provides the actual body of the function.

Function Declaration

- A function declaration tells the compiler about a function's name, return type, and parameters.
 - Also called function prototype
- Function prototype syntax:
`<type> <function name>(<type list>);`
 - `<type>` is the type of the return value (can be `void`)
 - `<type list>` is the list of types of input variables (can be empty if there is no input)

```
float averageGrade(int, int);
```

Function Definition

- The **function definition** can be placed anywhere in the program **after the function prototypes**.
- Function definition syntax:

```
<type> <function name>(<formal parameter list>){  
    <local declarations>  
    <sequence of statements>  
}
```

Function Definition Example

```
int sum(int operand1, int operand2)
{
    int s;
    s = operand1 + operand2;
    return s;
}
```

```
void printASCIICode(char code)
{
    printf("%c \s ASCII code is %d", code, code);
    return;
}
```

Function Call

- A function can be called to finish a task
- Function call syntax:

`<function name> (<actual parameter list>)`

- If the function definition is placed before it is called
 - No function prototype is needed
- If the function definition is placed after it is called
 - The function prototype is required

Function Call Example

```
void printASCIIcode(char);  
  
int main()  
{  
    printASCIIcode('A');  
    printASCIIcode('C');  
    printASCIIcode('F');  
}  
  
void printASCIIcode(char code)  
{  
    printf("%c \s ASCII code is %d\n", code, code);  
    return;  
}
```

Function declaration

Function calls

Function definition

The diagram illustrates the components of a C program. It shows a function declaration at the top, followed by a main function containing three calls to printASCIIcode. Below the main function is the actual function definition. Arrows point from the declaration line to the label 'Function declaration', from the three call lines to the label 'Function calls', and from the definition line to the label 'Function definition'.

Defining and Using a Function

- First, declare a function using a function prototype
 - `<type> name(<type1>, <type2>, ..., <typen>);`
- Second, define the function
 - `<type> name(<type1 p1>, <type2 p2>, ..., <typen pn>)`
- Third, call the function
 - `name(v1, v2, ..., vn);`

The Structure of C Programs

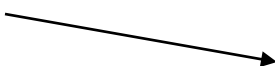
- Part 1: preprocessing statements
- Part 2: function prototypes
- Part 3: main() function
- Part 4: user-defined functions

The Structure of C Programs

```
#include <stdio.h>
```

```
int absolute(int);
```

Function declaration



```
int main()
```

```
{
```

```
    int value, answer;
```

```
    scanf("%d", &value);
```

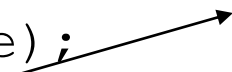
```
    answer = absolute(value);
```

```
    printf("The absolute value is %d.\n", answer);
```

```
    return 0;
```

```
}
```

Function call



```
int absolute(int x) {
```

```
    if (x >= 0)
```

```
        return x;
```

```
    else
```

```
        return -x;
```

```
}
```

Function definition



Library Functions

- The standard C library (18 Standard C Headers) contains a large collections of functions that can be called from a C program
 - math.h
 - stdio.h
 - stdlib.h
 - string.h
 - time.h
 -

Library Functions (Cont.)

- Three steps for using library functions
 - Find the header file which includes the library function to call
 - Use include directive *#include* to encompass the header file
 - Call the function in the program

```
#include<stdio.h>
#include<math.h>
int main() {
    double x = sqrt(2.0); //sqrt: declared in math.
    printf("x = %f", x); //printf: declared in stdio.h
    return 0;
}
```

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- **Array and Pointers**
- String
- Structure
- Dynamic Memory Allocation
- Recursion

Array

- An array is composed of a series of elements of one data type
- Array declaration tells the compiler
 - how many elements the array contains
 - what type is for these elements

```
float candy[365]; /* array of 365 floats */  
char code[12];    /* array of 12 chars */  
int grade[50]; /* array of 50 ints */  
.....
```

Array Elements

- To access elements in an array, you identify an individual element by using its subscript number, also called its **index**
- Note: The first element in an array has the index 0
 - `grade[0]` is the first element of the grade array
 - `grade[49]` is the 50th and last element

Array Initialization

- Three ways to initialize an array:

```
int grade[4];  
  
grade[0] = 10;  
grade[1] = 20;  
grade[2] = 30;  
grade[3] = 40;
```

```
int grade[4] = {10, 20, 30, 40 };
```

```
int grade[] = {10, 20, 30, 40 };
```

Array Initialization Example

```
int main() {  
    int a[5];  
    int i;  
    for (i = 0; i < 5; i++)  
        a[i] = i;  
    for (i = 0; i < 5; i++)  
        printf("a[%d] = %d\n", i, a[i]);  
    return 0;  
}
```

- What is the output of this program?
- Can we change $i < 5$ to $i \leq 5$?

Multi-Dimensional Arrays

- Arrays in C programs can have virtually as many dimensions as you want
- Declaration is accomplished by adding additional subscripts when it is defined
- E.g., `int table[4][3];`
 - defines a two-dimensional array
 - table is an array of table[0], table[1], table[2], and table[3]

Initializing An Array

- Three ways to initialize a multi-dimensional array:

```
int grade[2][3];  
  
grade[0][0] = 10;  
grade[0][1] = 20;  
grade[0][2] = 30;  
grade[1][0] = 40;  
grade[1][1] = 50;  
grade[1][2] = 60;
```

```
int grade[2][3] = {10, 20, 30, 40, 50, 60};
```

```
int grade[2][3] = {{10, 20, 30}, {40, 50, 60}};
```

Initializing An Array

```
#include<stdio.h>
#include<stdlib.h>
int main ()
{
    srand((unsigned)time(NULL));
    int random[2][2];
    int i, j;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            random[i][j] = rand() % 2;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 2; j++)
            printf("%c", random[i][j] ? 'y':'n');
        printf("\n");
    }
    return 0;
}
```

String

- A string is an **array of chars**
- E.g., `char s[10];`
 - `s` is a string which can store at most 9 characters
 - Terminated by a null character `'\0'`

```
int main () {  
    char word[20];  
    word[0] = 'H';  
    word[1] = 'e';  
    word[2] = 'l';  
    word[3] = '\1';  
    word[4] = 'o';  
    word[5] = '\0';  
    printf("The word is %s\n", word );  
    return 0;  
}
```

Memory

- The computer's memory is a sequential collection of “storage cells”
- Each cell can store one byte
- Each cell has an address
- When a normal variable is assigned a value, the value is put in the cells allocated to this variable
- Three elements:
 - **value**: mail
 - **variable**: owner
 - **address**: box no.

Memory (Cont.)

```
int m = 1;
```

00000000	00000000	00000000	00000001

0x377a

1 is the value of the variable **m**. It is put in the cells starting from address **0x377a**

The **postal card** is owned by **Judy**. It is put in the mailbox **303**

Address of Variables

- The operator `&` can be used to access the address of a variable
 - `int m;` then `&m` indicates the address of variable `m`
 - `int grade[10];` then `&grade[5]` indicates the address of `grade[5]`
- The array name can be directly used as the starting address of an array
 - `int grade[10];` `grade` is the address of the first element, so `grade` is equivalent to `&grade[0]`.

Address of Variables (Cont.)

```
int m;  
int grade[10];  
  
printf("Please input the value of m:");  
scanf("%d", &m);  
  
printf("m = %d is stored at address %p \n", m, &m);  
printf("grade[2] is stored at address %p \n",  
    &grade[2]);  
printf("grade[0] is stored at address %p \n",  
    &grade[0]);  
printf("the starting address of the array is %p \n",  
    grade);
```

Pointer

- A **pointer** is a variable whose value is the **address** of another variable, i.e., direct address of the memory location
- A pointer variable's value is the memory address of the **pointee** variable

Declare a Pointer

- Like any variable or constant, you must **declare a pointer** before using it to store any variable address.
- The general form of a pointer variable declaration is:
Type* var-name;
 - type is the pointer's base type; it must be a valid C data type
 - var-name is the name of the pointer variable
 - the asterisk * used to declare a pointer

```
int* ip;      /* pointer to an integer */
double* dp;   /* pointer to a double */
float* fp;    /* pointer to a float */
char* ch      /* pointer to a character */
```

Initialize a Pointer

- Like other variables, initialize pointers before using them
- To initialize the pointer, assign the address of a variable to this pointer

```
int main()
{
    int x;
    int* p;

    scanf("%d", p); /*incorrect, p is not initialized */

    p = &x;
    scanf("%d", p); /* correct, p points to x */
}
```

Pointer and Pointee

```
int x =3;  
int* p = &x;
```

00000000	00000000	00000000	00000011
0xbf	0xbc	0x75	0x8c

Address

0xbfbcb758c

0xbfbcb7590

- p is the pointer, p points to x, x is the pointee
- p's value is x's address.
- p is also allocated cells to store its value

Exercise

```
int main()
{
    int x = 3;
    int* p = &x;

    printf("x = %d\n", x);
    printf("The address of x is %p \n", &x);
    printf("The value of p is %p \n", p);
    printf("The address of p is %p \n", &p);
    return 0;
}
```

Output?

x
p

00000000	00000000	00000000	00000011
0xbf	0xbc	0x75	0x8c

0xbfbc758c

0xbfbc7590

Dereferencing

- &pointee: pointee's address
- *pointer: value stored in the memory location pointed by the pointer
 - *pointer is called dereferencing

```
int x = 3;
int* p = &x;
printf("x = %d\n", x);
printf("x = %d\n", *p);

*p = 4;
printf("x = %d\n", *p);
printf("x = %d\n", x);
```

Output?

Example

```
int main()
{
    int x, *p;
    p = &x;          /* initialise pointer */

    *p = 0;           /* set x to zero */
    printf("x is %d\n", x);
    printf("*p is %d\n", *p);

    *p += 1;          /* increment what p points to */
    printf("x is %d\n", x);

    (*p)++;           /* increment what p points to */
    printf("x is %d\n", x);
    return 0;
}
```

Exercise

```
int main()
{
    int a = 100, b = 88, c = 8;
    int *p1 = &a, *p2, *p3 = &c;

    p2 = &b;    // p2 points to b
    p2 = p1;    // p2 points to a
    b = *p3;    // assign c to b
    *p2 = *p3;   // assign c to a
    printf("%d%d%d", a, b, c);
}
```

What is the output?

Exercise

```
int main()
{
    int value1 = 5, value2 = 15;
    int *p1, *p2;

    p1 = &value1; //p1 points to value1
    p2 = &value2; //p2 points to value2

    *p1 = 10;      //assign 10 to value1
    *p2 = *p1;     //assign v1 to v2

    p1 = p2;      // p1 points to value2
    *p1 = 20;     // assign 20 to value2

    printf("%d %d", value1, value2);
}
```

What is the output?

Return Values Through Pointers

```
double calSumAverage(double, double, double* );
int main( )
{
    double x = 1.0, y = 2.0;
    double average, sum;
    sum = calSumAverage(x , y, &average );
    printf("The sum is %d, the average is %d", sum, average);
    return 0;
}

double calSumAverage(double no1, double no2, double
    *pAverage)
{
    double sum;
    sum = no1 + no2;
    *pAverage = sum / 2;
    return sum;
}
```

Compare Two Cases

```
int main( )
{
    .....
    calSumAverage( &average );
    .....
}

double calSumAverage( double *pAverage )
{
    .....
    *pAverage = sum / 2;
    .....
}
```

Pass the address



The value of **average** is changed after the function call

Compare Two Cases

```
int main( )
{
    .....
    calSumAverage(average) ;
    .....
}

double calSumAverage(double pAverage)
{
    .....
    pAverage = sum / 2;
    .....
}
```

Pass the value



The value of `average` is **NOT** changed after the function call

The only way a function can access another function's local variables:

POINTERS AS PARAMETERS

Pointer to Pointer

```
int x = 58;  
int* p ;  
int** q;  
  
p = &x;  
q = &p;  
  
printf("%d %d %d", x, *p, **q);
```

x	00000000	00000000	00000000	00111010	0xbfbcb758c
p	0xbf	0xbc	0x75	0x8c	0xbfbcb7590
q	0xbf	0xbc	0x75	0x90	0xbfbcb7594

Array

- An array name indicates the address of the first element

```
void main()  
{  
    int a[5] = {2, 4, 6, 8, 22};  
    printf("%d %d %d", *a, a[0], *(&a[0]));  
}
```

What is the output?

Pointer and Array

- Given a pointer p , $(p + n)$ points to the address $p + nk$ where k is size of data type pointed by p in the declaration
 - $*(p + n)$ is same as $a[n]$ if $p = \&a[0]$

```
int a[5] = {2, 4, 6, 8, 22};  
int *p;  
p = &a[0];
```

a[5] {	0000000	0000000	0000000	00000010	← p
	0000000	0000000	0000000	00000100	← p + 1
	0000000	0000000	0000000	00000110	← p + 2
	0000000	0000000	0000000	00001000	← p + 3
	0000000	0000000	0000000	00010110	← p + 4

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- **String**
- Structure
- Dynamic Memory Allocation
- Recursion

String

- A string is an array of chars terminated with a null character, '\0'
- String declaration
 - `char str[10];`
 - This is a string that contains at most 9 characters

String Initialization

- There are several ways to initialize a string
 - `char str[20] = { 'H', 'e', 'l', 'l', 'o', '\0' };`
 - `char str[20] = "Hello";`
 - `'\0'` is automatically set to `str[5]`
 - `char str[] = "Hello";`
 - `str`'s size is 6
 - `char str[20]; str = "Hello";`
 - Exception: array type 'char [20]' is not assignable

Output String

- To output a string to the screen, two functions can be used
 - `printf`
- Defined in `stdio` library, so we need to include `stdio.h` in the program

The printf Function

- `%s` must be used in the format string to indicate a string is to be printed
- `printf` stops printing when it meets `'\0'`

```
char str[] = "This is a message";  
printf("%s", str);
```

The puts Function

- Simpler than `printf`
- `puts` function outputs a string to standard output,
 - stops printing when it meets `'\0'`

```
char str[] = "This is a message";  
puts(str);
```

Input String

- To input a string from the keyboard, two functions can be used
 - `scanf`
- Defined in `stdio` library, so we need to include `stdio.h` in the program

The scanf Function

- `%s` or `%ns` is used in the format string to indicate a string is to be read
 - `%s`: scans up to but not include the next space character

```
char str1[20], str2[20], str3[20];  
scanf("%s%s%s", str1, str2, str3);
```

Assume the input is: Hello World !

Then:

- `str1`: Hello
- `str2`: World
- `str3`: !

The scanf Function (Cont.)

- `%s` or `%ns` is used in the format string to indicate a string is to be read
 - `%ns`: scans the next `n` characters or up to but not include the next space character, whichever comes first

```
char str1[20], str2[20], str3[20];  
scanf("%2s%3s%5s", str1, str2, str3);
```

Assume the input is: Hello World !

Then:

- `str1`: He
- `str2`: llo
- `str3`: World

The fgets Function

- fgets() gets a line from the standard input
- Be careful, do not overflow the string buffer (exceed the size of array)
 - fgets(your_line, sizeof(your_line), stdin)

```
char input[5];  
fgets(input, sizeof(input) / sizeof(char), stdin);  
puts(input);
```

String Functions

- Common string functions include functions
 - computing the length of a string (`strlen`)
 - copying strings (`strcpy`, `strncpy`)
 - comparing strings (`strcmp`, `strncmp`)
 - concatenating strings (`strcat`)
 - more ...
- To use these functions, we must include the file `string.h`, e.g., `#include <string.h>`

String to Number Functions

- Some functions can transfer number string to numbers
 - atoi
 - atol
 - atof
- If use these functions, must include [stdlib.h](#), e.g.,
`#include<stdlib.h>`

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- **Structure**
- Dynamic Memory Allocation
- Recursion

Basic Idea

- To represent one item, we can declare single **variable**
 - E.g., `int age;`
- To represent several items of the same type, we can declare an **array**
 - E.g., `int age[100];`
- To represent several items of different types, we can declare a **structure**
 - E.g., we have 100 students, each student's information includes name, age, gender, major, how can we do?

Example

- A birthday consists of 3 parts: year, month and day.
- We can define them in this way

```
int year = 2007;  
int month = 11;  
int day =13;
```
- These 3 variables are logically related, we cannot see that the above declarations are for one date
- It would be better if they can be grouped **together**.
- The **structure** in C helps.

Example (Cont.)

A structure called **date**

```
struct date {  
    int year;           // 1st member variable  
    int month;          // 2nd member variable  
    int day;            // 3rd member variable  
};                      // don't forget the semicolon !
```

date now is a new type that can be used like int, char, ...
date has three members.

struct Variable Declaration

There are three ways to declare a variable of struct type

```
struct date {  
    int year;  
    int month;  
    int day;  
};  
  
struct date birthday;
```

```
struct date {  
    int year;  
    int month;  
    int day;  
} birthday;
```

```
typedef struct {  
    int year;  
    int month;  
    int day;  
} date;  
  
date birthday;
```

struct Member Accessing

- A member of a structure is accessed by specifying the **variable name**, followed by a **period**, and then the **member name**

```
struct date today;  
  
scanf("%d %d", &today.month, &today.day);  
if(today.month == 1 && today.day == 1)  
    printf("Happy new year!");
```

struct-to-struct Assignment

```
struct studentRecord{
    char name[15];
    int id;
    char dept[5];
    char gender;
};

struct studentRecord student1, student2;

strcpy(student1.name, "Tom Hanks");
student1.id = 12345;
strcpy(student1.dept, "COMP");
student1.gender = 'M';

student2 = student1;
```

```
strcpy(student2.name, student1.name);
student2.id = student1.id;
strcpy(student2.dept, student1.dept);
student2.gender = student1.gender;
```

Structures in Functions

- A structure parameter value can be passed to a function
- A structure can be returned

```
struct date nextDay(struct date today)
{
    struct date tomorrow;
    // code to calculate the date of tomorrow
    .....
    return tomorrow;
}
```

Nested Structures

- We can nest structures inside structures

```
struct point{  
    double x, y;  
};  
struct point p;  
  
struct line{  
    struct point p1, p2;  
};  
struct line l;  
  
struct triangle{  
    struct point p1, p2, p3;  
};  
struct triangle t;
```

Assessing the members:

p.x

p.y

l.p1.x

l.p1.y

l.p2.x

l.p2.y

Pointer to Structures

- You can define pointers to structures in the same way as you define pointer to any other variable
 - `struct date *date_pointer;`
- To find the address of a structure variable, place the '&'; operator before the structure's name
 - `date_pointer = &today;`
- To access the members of a structure using a pointer to that structure, you must use the '->' operator
 - `date_pointer -> day;`

Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- Structure
- **Dynamic Memory Allocation**
- Recursion

Memory Allocation

- Two ways
 - static memory allocation
 - Memory is allocated invisibly when a variable is declared
 - e.g., `int i;` four bytes are allocated to store the value of `i`
 - dynamic memory allocation
 - Memory is allocated visibly by calling some allocation functions.

Static Memory Allocation

- Memory allocation determined **at compiling time**:
 - `int x;` // 4 bytes
 - `char y;` // 1 byte
 - `float a;` // 4 bytes
 - `double b;` // 8 bytes
 - `int a[10];` // 10*4 bytes
- The type of the variable determines how much memory the compiler allocates

Static Memory Allocation (Cont.)

- If too much memory is allocated and then not used, there is a **waste of memory**
- If **not enough memory is allocated**, the program is not able to handle the input data

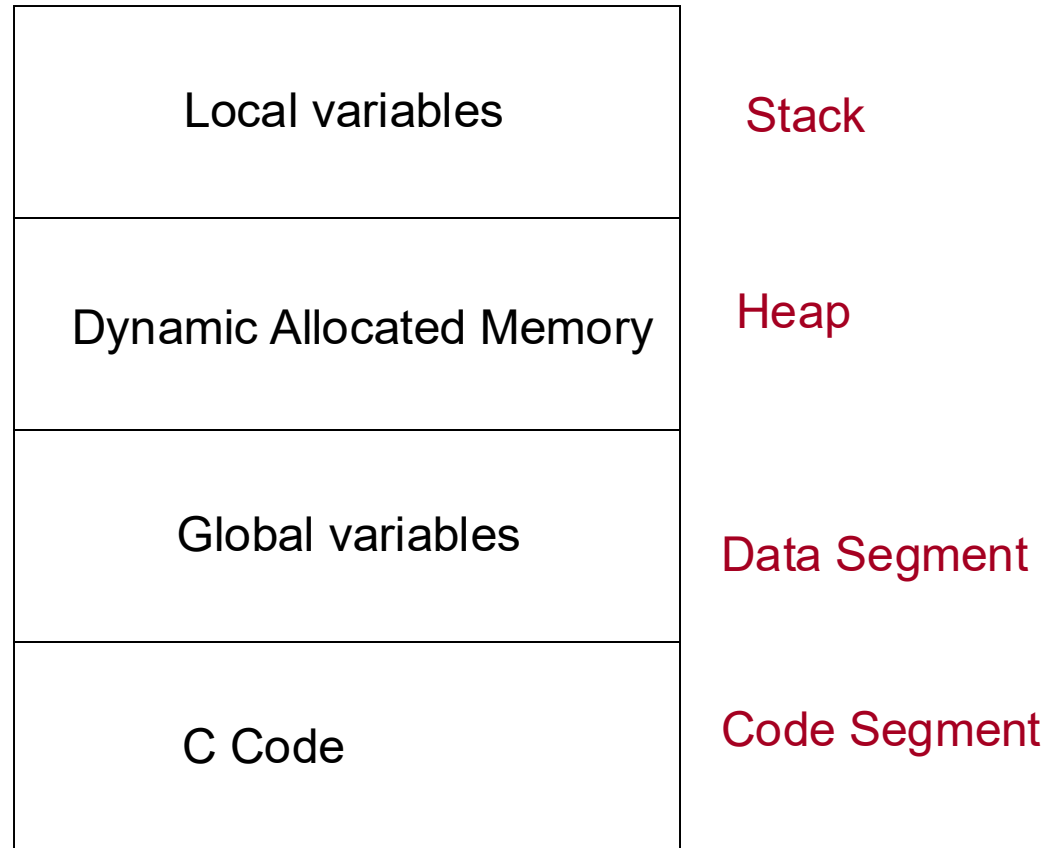
```
int grade[100];
int i = 0;
scanf("%d", &grade[i]);
while(grade[i] != -1)
{
    printf("%d\n", grade[i]);
    i++;
    scanf("%d", &grade[i]);
}
```

What if there are only **5** grades?
What if there are **1000** grades?

Dynamic Memory Allocation

- Dynamic memory allocation allocates memory at **execution time** or **runtime** when needed, **free the memory** when the memory is not needed
- **Dynamic memory** is allocated in the **heap** (also called the free storage, a large pool of unused memory area) by the system.
- **Static memory** is allocated in the **stack** (last-in-first-out) the system.

Memory Allocation Process



Dynamic Memory Allocation

- Two basic functions:
 - `void* malloc(size_t size)`
 - Allocates a block of memory of specified size and returns a pointer of type void
 - We can assign it to a pointer of any type
 - A null pointer is returned if there is not enough space in the heap
 - `void free(void* ptr)`
 - releases the used memory when it is no longer needed
- To use these functions, the `stdlib.h` header file must be included.

Example 1

```
int* ptr;  
ptr = (int*)malloc(sizeof(int));  
if (ptr != NULL)  
    *ptr = 23;  
printf("Value stored is %d\n", *ptr);  
free(ptr);
```

Example 2

```
#include <stdio.h>
#include <stdlib.h>
struct stdrec {
    char name[20];
    int id;
};
int main(){
    struct stdrec * p;
    printf("***Dynamic Memory Allocation***\n\n");
    p = (struct stdrec *) malloc(sizeof(struct stdrec));
    if( p != NULL){
        printf("Student Name: ");
        scanf("%s",p->name);
        printf("Student ID: ");
        scanf("%d", &p->id);
        printf("Student Name: %s ", p->name);
        printf("ID: %d\n", p->id);
    } else
        printf("Out of Memory\n");
    free(p);
}
```

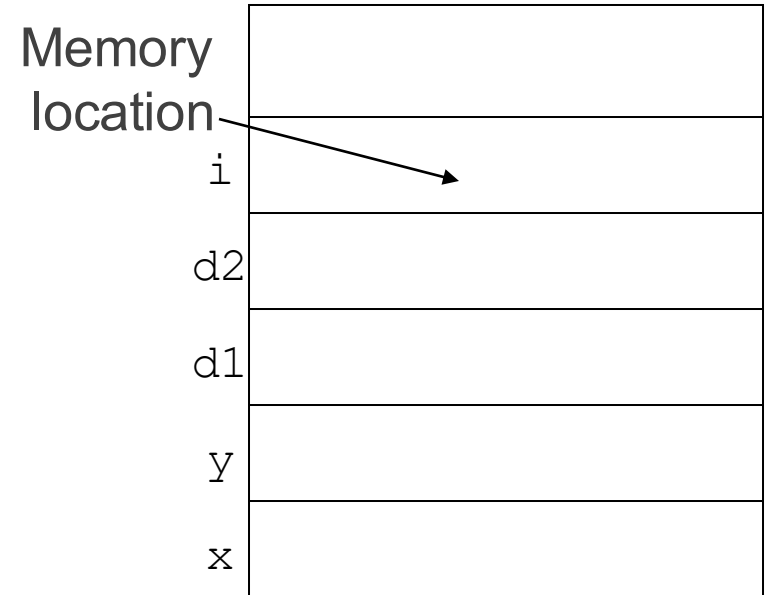
For more details of "scanf" and "printf", please refer to:
<https://www.runoob.com/cprogramming/c-function-printf.html>
<https://www.runoob.com/cprogramming/c-function-scanf.html>

Memory Leak

- Memory leak happens when memory is allocated but not released
- It will cause an application to gradually consume memory then the available memory for other applications is reduced.
- The allocated memory must be freed!

Functions & Memory

- Every function needs a place to store its **local variables**
Collectively, this storage is called the **stack**
- Each storage space has a numerical address
- Instead of using raw addresses, we use **variables** to attach a name to an address
- All of the data/variables for a particular function call are located in a **stack frame**

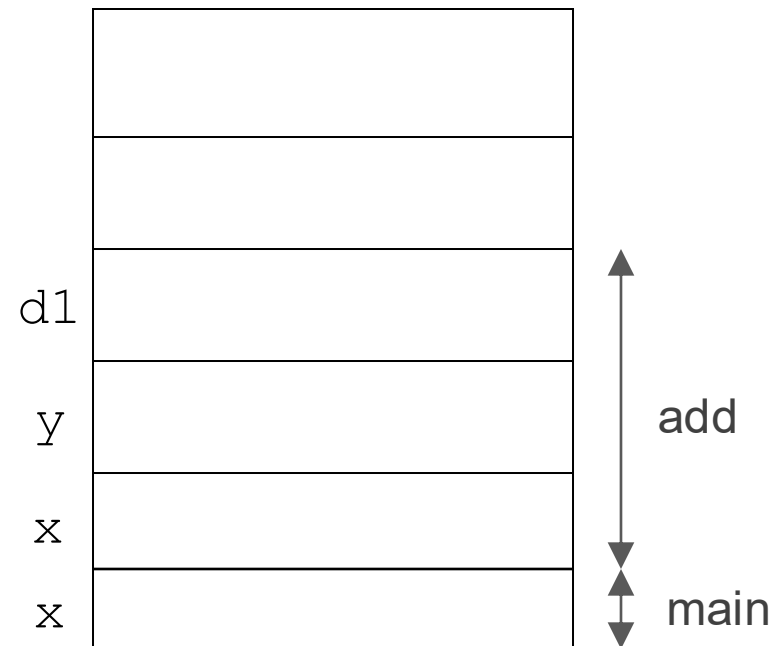


```
void aFunc(int x, int y)
{
    double d1, d2;
    int i;
}
```

Functions & Memory (Cont.)

- When a function is called, a new stack frame is created
- Parameters and return values are passed *by copy* (ie, they're copied into and out of the stack frame)
- When a function finishes, its stack frame is *reclaimed*

```
void add(int x, int y) {  
    double d1 = x + y;  
}  
  
int main() {  
    int x = 7;  
    add(1, 2);  
    add(2, 3);  
    return 0;  
}
```



Outline

- Introduction to C
- Primary data types and variables
- Formatted IO
- Function
- Array and Pointers
- String
- Structure
- Dynamic Memory Allocation
- Recursion

Recursive Functions

- A recursive function
 - calls itself
 - uses different parameter values
 - stops calling itself when the base case is met
- Recursive functions are commonly used in the applications in which the **solution to a problem** can be expressed in terms of **successively applying the same solution** to subsets of the problem
- The famous factorial calculation problem
 - $n! = n * (n-1) * (n-2) * \dots * 1$
 - $n! = n * (n-1)!$ and $1! = 1$
 - recursive step
 - base case

Recursive Functions (Cont.)

Recursive step: the result of `fac(n)` is `n * fac(n - 1)`

Base case: `fac(1)` is 1

```
int fac(int n) // Assume n >= 0
{
    int product;

    if (n <= 1)
        return 1;

    product = n * fac(n-1);
    return product;
}
```

Base case

Recursive
step

Recursive Functions (Cont.)

Assume in main program, we use `fac(3)` to call the `fac` function

```
int fac(int n)
{
    int product;
    if(n <= 1)
        return 1;
    product = n * fac(n-1);
    return product;
}
```

```
int main()
{
    .....
    r = fac(3);
    .....
}
```

fac(3) :
3 <= 1 ?
product = 3 * fac(2)
return product

fac(1)
1 <= 1?
return 1

fac(2) :
2 <= 1 ?
product = 2 * fac(1)
return product

Base Case

- The recursion continues until some condition (termination condition) is met
- Base case is also called stop condition
- Always write the termination condition and make sure that the condition is reachable.
- Otherwise the recursion **WILL NOT STOP!**

Will this recursion stop?

```
#include <stdio.h>

int recurse(int i)
{
    return recurse(i-1);
}

int main()
{
    int i;
    scanf("%d", &i);
    recurse(i);
    return 0;
}
```

Will this recursion stop?

```
#include <stdio.h>

int recurse(int i)
{
    if (i<=0)
        return 0;
    return recurse(i-1);
}

int main()
{
    int i;
    scanf("%d", &i);
    recurse(i);
    return 0;
}
```

Will this recursion stop?

```
#include <stdio.h>

int recurse(int i)
{
    if(i==0)
        return 0;
    return recurse(i-1);
}

int main()
{
    int i;
    scanf("%d", &i);
    recurse(i);
    return 0;
}
```